

Consola Retro SDK — Interfaz Gráfica y Experiencia de Usuario

Fundamentos de Sistemas Embebidos

Rosales Vigil Karla Sofia - 317309812

Tutorial individual — Módulos: pantalla.py · arranque.py · galeria.py

1. Objetivos

1.2 General:

Implementar y comprender la interfaz gráfica de una consola de videojuegos retro embebida en una Raspberry Pi 4. Se adquirirá el conocimiento para configurar pygame sobre Raspberry Pi OS Lite (sin entorno gráfico de escritorio), a crear una animación de arranque con efecto de fundido de entrada y salida, y a construir un menú de galería navegable por gamepad que muestra el catálogo de ROMs disponibles.

1.3 Específico:

Desarrollar un sistema unificado de interfaz gráfica (UI) y experiencia de usuario (UX) para la “Consola Retro SDK” mediante la inclusión de los módulos *pantalla.py*, *arranque.py* y *galería.py*, logrando así un entorno visual intuitivo, navegación en catalogo de juegos de 8 a 16 bits, construir un menú de galería navegable por gamepad que muestra el catálogo de ROMs disponibles, una animación de arranque con efecto de fundido de entrada y salida, operando de manera nativa sobre un sistema operativo base.

2. Lista de materiales

Tabla 1. Hardware requerido

Componente	Notas
Raspberry Pi 4 Model B (2 GB o más)	Mínimo 2 GB RAM recomendados
Tarjeta microSD (16 GB o más, clase 10)	Clase 10 o UHS-I para velocidad
Monitor o televisor con entrada HDMI	Resolución mínima 720p
Cable HDMI a micro-HDMI	Incluido en algunos kits
Fuente de alimentación 5 V / 3 A USB-C	Oficial Raspberry Pi recomendada
Gamepad USB o Bluetooth compatible	Compatible con SDL2 (ej. Xbox, genérico)
Memoria USB (FAT32 o exFAT)	Para carga de ROMs adicionales
Computadora con acceso a internet	Para preparar la tarjeta SD

En cuanto al software, se requiere:

- Raspberry Pi OS Lite (64-bit), sin entorno de escritorio
- Python 3.11 o superior
- pygame 2.x (instalado vía apt: python3-pygame)

- SDL2 con drivers kmsdrm para acceso directo al framebuffer
- ROMs libres de derechos para NES, SNES y Game Boy Advance
- pyudev — bindings Python para libudev (python3-pyudev)
- mednafen — emulador multi-sistema (apt install mednafen)

3. Antecedentes teóricos

3.1 Sistemas embebidos y salida de video sin escritorio

Un sistema embebido es un sistema computacional de propósito específico integrado dentro de un dispositivo más grande. A diferencia de una computadora de propósito general, el sistema embebido ejecuta una sola aplicación o un conjunto reducido de aplicaciones de forma continua y autónoma desde el momento en que recibe alimentación eléctrica [1].

En sistemas Linux de propósito específico es común prescindir del entorno de escritorio (X11 o Wayland) para reducir el consumo de memoria RAM, el tiempo de arranque y la complejidad del sistema. En este escenario, la salida gráfica debe realizarse directamente a través del framebuffer del kernel o mediante una API de bajo nivel como KMS/DRM (Kernel Mode Setting / Direct Rendering Manager).

SDL2 (Simple DirectMedia Layer 2) es una biblioteca multiplataforma que abstrae el acceso al hardware de video, audio y entrada. En Raspberry Pi OS Lite, SDL2 puede configurarse para utilizar el driver kmsdrm, que permite escribir directamente en el framebuffer sin necesidad de un servidor X [2]. La variable de entorno `SDL_VIDEODRIVER=kmsdrm` indica a SDL2 que debe usar este driver.

3.2 Pygame y el modelo de doble buffer

Pygame es una colección de módulos Python construida sobre SDL2. Proporciona una API de alto nivel para manejo de ventanas, eventos, gráficos 2D y audio. El modelo de renderizado de pygame sigue el paradigma de doble buffer: toda operación de dibujo se realiza sobre una superficie en memoria (back buffer), y únicamente cuando se llama a `pygame.display.flip()` el contenido se transfiere a la pantalla visible (front buffer). Esto elimina el parpadeo (tearing) que se produciría si se dibujara directamente sobre el buffer visible [3].

El ciclo básico de renderizado sigue la siguiente estructura:

```
while True:
    # 1. Limpiar buffer
    pantalla.fill((0, 0, 0))
    # 2. Dibujar elementos
    pantalla.blit(imagen, (x, y))
    # 3. Actualizar display
    pygame.display.flip()
    # 4. Controlar FPS
    reloj.tick(30) 0
```

3.3 Renderizado de texto con fuentes bitmap y SysFont

pygame permite renderizar texto mediante la clase `pygame.font.Font` (fuente TTF personalizada) o `pygame.font.SysFont` (fuente del sistema). La función `render()` devuelve una superficie con el texto dibujado, que luego se transfiere a la pantalla con `blit()`. El parámetro `antialias=True` suaviza los bordes del texto mediante interpolación de colores [3].

3.4 Animación por interpolación de alpha

El efecto de fundido (fade) se logra modificando el canal alfa (transparencia) de una superficie de pygame. El método `Surface.set_alpha(valor)` acepta valores entre 0 (completamente transparente) y 255 (completamente opaco). Para un fundido de entrada durante T segundos a F fotogramas por segundo, el incremento de alpha por frame es $255 / (T \times F)$. Se genera una copia del logo en cada frame con `Surface.copy()` para evitar modificar la superficie original [3].

3.5 Ventana deslizante para menús largos

Cuando el número de elementos del menú supera el espacio disponible en pantalla, se utiliza una ventana deslizante (sliding window). Dados el índice del elemento seleccionado y el número máximo de elementos visibles V, la ventana se calcula como:

```
inicio = max(0, indice - V // 2)
fin    = min(total, indice + V)
```

Esto centra el elemento seleccionado en la lista siempre que sea posible, y ancla la ventana al inicio o al final cuando el índice se acerca a los extremos.

3.6 Gestión del framebuffer entre procesos

En Raspberry Pi OS Lite el acceso al display se realiza mediante KMS/DRM, que garantiza acceso exclusivo al framebuffer: solo un proceso puede tener el control del display en un momento dado [4]. Cuando pygame está activo, tiene el framebuffer. Al lanzar mednafen, si pygame no libera el framebuffer antes, mednafen no puede inicializar su propio display y falla. La solución es llamar a `pygame.display.quit()` antes de lanzar el emulador y reinicializar pygame al regresar.

3.7 Gestión de dispositivos USB con pyudev

udev es el gestor de dispositivos del kernel Linux. Cuando el kernel detecta una memoria USB, genera un evento `uevent` que pyudev puede escuchar desde Python mediante un monitor asíncrono, filtrando por subsistema 'block' y tipo 'partition'. La ventaja sobre el polling es que el hilo de monitoreo queda bloqueado hasta que el kernel genera un evento, sin ciclos de CPU desperdiciados [3].

4. Advertencias y cuidado de componentes

4.1 Seguridad eléctrica

- La Raspberry Pi opera a 5 V CC. Aunque el voltaje es seguro, una fuente de alimentación de mala calidad puede dañar permanentemente la tarjeta por sobretensión. Se recomienda utilizar únicamente la fuente oficial o equivalentes certificadas.
- Desconecte la alimentación antes de conectar o desconectar periféricos en los pines GPIO.
- Nunca conecte cables a los pines GPIO mientras el sistema está encendido sin asegurarse de los voltajes y configuraciones correctas.

4.2 Cuidado de la tarjeta microSD

- La tarjeta microSD es el componente más frágil del sistema. Los fallos de escritura pueden corromper el sistema de archivos. Para minimizar este riesgo, el sistema está configurado para apagarse siempre mediante shutdown antes de cortar la alimentación.
- Evitar retirar la tarjeta microSD con el sistema encendido.
- Realice respaldos periódicos de la imagen de la tarjeta con herramientas como dd o balenaEtcher.

4.3 Temperatura de operación

- La Raspberry Pi 4 puede alcanzar temperaturas elevadas bajo carga sostenida. Se recomienda el uso de un disipador de calor pasivo o activo para garantizar la estabilidad del sistema a largo plazo.
- La temperatura máxima de operación especificada es 85 °C. Por encima de esta temperatura el procesador reduce su frecuencia automáticamente (throttling).

5. Descripción de componentes de software

El subsistema de interfaz gráfica está compuesto por tres módulos Python independientes que se comunican únicamente a través de la referencia al objeto Pantalla.

Tabla 2. Módulos del subsistema de interfaz gráfica

Módulo	Archivo	Función principal
Pantalla	pantalla.py	Renderizado, fuentes y audio con pygame
Arranque	arranque.py	Animación fade-in/out del logotipo
Galería	galeria.py	Menú principal y catálogo de juegos
Entrada	entrada.py	Abstracción de gamepad y teclado a acciones lógicas
Emulador	emulador.py	Lanzamiento de mednafen y gestión del framebuffer
Monitor USB	usb_monitor.py	Detección, montaje y copia de ROMs desde USB
Principal	main.py	Punto de entrada; instancia y conecta todos los módulos

5.1 Módulo pantalla.py — Clase Pantalla

La clase Pantalla encapsula la inicialización de pygame y todas las operaciones de renderizado. Este diseño sigue el principio de responsabilidad única: ningún otro módulo del sistema llama a pygame directamente, lo que permite cambiar la biblioteca de renderizado en el futuro sin modificar el resto del sistema.

Al instanciar la clase se inicializa pygame, se configura el mezclador de audio (pygame.mixer), y se crea la ventana en modo pantalla completa. Se definen tres tamaños de fuente monoespaciada para uso en todo el sistema: grande (52 pt), normal (32 pt) y pequeña (20 pt). También se oculta el cursor del ratón, innecesario en un sistema controlado por gamepad.

5.1.1 Métodos principales

Tabla 3. Métodos del subsistema de interfaz

Método	Clase	Descripción
--------	-------	-------------

<code>limpiar(color)</code>	Pantalla	Rellena la superficie con un color sólido
<code>dibujar_texto(...)</code>	Pantalla	Renderiza texto con fuente, color y alineación
<code>dibujar_imagen(...)</code>	Pantalla	Carga y escala una imagen PNG en pantalla
<code>dibujar_rectangulo(...)</code>	Pantalla	Dibuja rectángulos rellenos o con borde
<code>reproducir_sonido(ruta)</code>	Pantalla	Reproduce un archivo WAV con <code>pygame.mixer</code>
<code>actualizar()</code>	Pantalla	Vuelca el buffer al display (<code>display.flip</code>)
<code>ejecutar()</code>	Arranque	Orquesta la animación de arranque completa
<code>_fade(logo, x, y, dir)</code>	Arranque	Anima la opacidad del logo frame a frame
<code>ejecutar()</code>	Galeria	Loop principal del menú a 30 fps
<code>_dibujar()</code>	Galeria	Renderiza título, separador y lista de ROMs
<code>confirmar_salida()</code>	Galeria	Pantalla de confirmación antes de apagar

5.2 Módulo `arranque.py` — Clase `Arranque`

La clase `Arranque` implementa la secuencia de presentación que se ejecuta una única vez al encender el sistema. Su propósito es establecer la identidad visual del producto y proporcionar retroalimentación al usuario de que el sistema ha arrancado correctamente.

La secuencia completa es la siguiente:

1. Se carga el archivo `logo_SDK.png` y se escala al 50 % del ancho de pantalla manteniendo la proporción original.
2. Se reproduce el archivo de sonido `entrada_coin.wav` mediante `pygame.mixer.Sound`.
3. Se ejecuta el fundido de entrada (`_fade` con dirección 'in') durante 1.5 segundos.
4. El logo permanece visible durante 2 segundos. Durante este tiempo el usuario puede presionar `Escape` para saltar la animación.
5. Se ejecuta el fundido de salida (`_fade` con dirección 'out') durante 1.5 segundos.
6. El control se devuelve al módulo principal para que inicie el menú de galería.

El método `_fade()` calcula el número total de pasos como $\text{duracion_fade} \times 60$ (asumiendo 60 fps). En cada iteración calcula el valor de `alpha` de forma lineal, crea una copia de la superficie del logo con ese valor de `alpha`, limpia la pantalla y dibuja el logo con la nueva opacidad.

5.3 Módulo `galeria.py` — Clase `Galeria`

La clase `Galeria` implementa el loop principal del sistema una vez concluida la secuencia de arranque. Su responsabilidad abarca el escaneo del catálogo de ROMs, la navegación por el menú, el lanzamiento del emulador, la gestión del apagado y la detección de nuevas ROMs introducidas por USB.

5.3.1 Escaneo del catálogo

El método `_escanear_roms()` recorre las carpetas `roms/nes`, `roms/snes` y `roms/gba` buscando archivos con las extensiones correspondientes (`.nes`, `.smc/.sfc` y `.gba`). Los resultados se almacenan en un diccionario indexado por consola y se convierten en una lista plana con `_aplanar_catalogo()` para facilitar la navegación con un índice entero.

5.3.2 Renderizado del menú

El método `_dibujar()` es responsable de todo el renderizado del menú. El fondo se rellena con el color (15, 15, 20), un azul oscuro que evita el deslumbramiento. El título se dibuja centrado en la parte superior. Una línea horizontal actúa como separador visual. La lista de ROMs se dibuja con la ventana deslizante descrita en la sección 3.5. Cuando el ítem es el seleccionado, se dibuja un rectángulo de color de selección (50, 150, 255) detrás del texto, que adopta entonces color negro para garantizar contraste. Las etiquetas de consola se intercalan entre grupos de ROMs.

5.3.3 Confirmación de apagado

Al presionar `Start` o `Escape`, el sistema muestra una pantalla de confirmación antes de ejecutar `sudo shutdown -h now`. Esta medida protege la tarjeta microSD de corrupciones por corte abrupto de energía. El bucle de confirmación procesa eventos directamente de la cola de `pygame` mediante `pygame.event.get()`, separado del loop principal de la galería.

5.4 Módulo `entrada.py` — Clase `Entrada`

La clase `Entrada` implementa el patrón de abstracción de entrada: en lugar de que el resto del sistema conozca los botones físicos del gamepad, todos los módulos de nivel superior trabajan con acciones lógicas (`ACCION_ARRIBA`, `ACCION_CONFIRMAR`, etc.). Esto hace que el sistema sea independiente del modelo de gamepad. Soporta conexión y desconexión en caliente mediante los eventos `JOYDEVICEADDED` y `JOYDEVICEREMOVED` de `pygame`, y aplica una zona muerta de 0.5 en los ejes analógicos para evitar navegación errática por drift.

Tabla 4. Mapeo de acciones lógicas a controles físicos

Acción lógica	Botón gamepad	Tecla teclado
ARRIBA	Hat ↑ / Eje Y < -0.5	↑
ABAJO	Hat ↓ / Eje Y > +0.5	↓
CONFIRMAR	Botón 0 (A)	Enter
ATRAS	Botón 1 (B)	Escape
INICIO	Botón 7 (Start)	F1

5.5 Módulo emulador.py — Clase Emulador

La clase Emulador gestiona todo el ciclo de vida de una sesión de juego. Oculta la complejidad de la gestión del framebuffer al resto del sistema: la galería simplemente llama a `emulador.lanzar(console, nombre, ruta)` y recupera el control cuando el juego termina. Antes de lanzar mednafen muestra una pantalla con los controles de la consola seleccionada. Soporta NES, SNES y GBA mediante el flag `-force_module` de mednafen.

5.6 Módulo usb_monitor.py — Clase MonitorUSB (resumen)

MonitorUSB detecta la inserción de memorias USB en tiempo real mediante `pyudev`, las monta en modo solo lectura, copia las ROMs nuevas al proyecto evitando duplicados (comparación insensible a mayúsculas), y desmonta el USB al terminar. Opera en un hilo `daemon` separado para no interferir con la interfaz gráfica; la comunicación con el loop principal se realiza a través de un booleano protegido por `threading.Lock` para evitar condiciones de carrera. Si hay un juego en curso al detectar el USB, termina el proceso de mednafen de forma controlada.

6. Configuración de la Raspberry Pi

6.1 Instalación del sistema operativo

Se utiliza Raspberry Pi OS Lite (64-bit) sin entorno gráfico de escritorio. La imagen se graba en la tarjeta microSD con balenaEtcher o Raspberry Pi Imager. Durante la configuración inicial se deben habilitar SSH y configurar usuario, contraseña y red Wi-Fi para facilitar la instalación remota de paquetes.

6.2 Variables de entorno SDL

Para que pygame utilice el driver `kmsdrm` y ALSA para audio, se deben establecer las siguientes variables de entorno en el archivo `~/.bashrc` del usuario:

```
export SDL_VIDEODRIVER=kmsdrm
export SDL_AUDIODRIVER=alsa
```

Estas mismas variables se configuran en el servicio `systemd` para garantizar que estén disponibles al arrancar automáticamente sin sesión de usuario activa.

6.3 Grupos de usuario

El usuario que ejecuta la aplicación debe pertenecer a los grupos `video`, `audio`, `input` y `render` para tener acceso al framebuffer, dispositivos de audio y controles de entrada sin privilegios de `root`:

```
sudo usermod -aG video,audio,input,render $USER
```

6.4 Grupos de usuario

mednafen se instala desde los repositorios oficiales de Raspberry Pi OS y se enlaza en `/usr/local/bin`:

```
sudo apt update && sudo apt install -y mednafen
sudo ln -sf /usr/games/mednafen /usr/local/bin/mednafen
```

6.5 Autoarranque con systemd

El script de instalación (`instalar.sh`) crea y habilita el servicio `console-retro.service` en `systemd`. El archivo de servicio incluye las variables SDL, define el directorio de trabajo y configura el reinicio automático en caso de fallo:

```

[Unit]
Description=Consola Retro SDK
After=multi-user.target sound.target

[Service]
Type=simple
User=pi
Environment=SDL_VIDEODRIVER=kmsdrm
Environment=SDL_AUDIODRIVER=alsa
ExecStart=/usr/bin/python3 /home/pi/consola_retro/src/main.py
Restart=always

[Install]
WantedBy=multi-user.target

```

6.6 Estructura de carpetas de ROMs

Las ROMs deben colocarse en subcarpetas según la consola dentro de src/roms/:

```

src/
  roms/
    nes/      ← archivos .nes
    snes/     ← archivos .smc o .sfc
    gba/      ← archivos .gba

```

7. Desarrollo de los módulos de interfaz

7.1 Clase Pantalla

A continuación, se muestra el fragmento central de la clase Pantalla, con la inicialización de pygame y los tres tamaños de fuente:

```

import pygame

class Pantalla:
    def __init__(self, ancho, alto, pantalla_completa=True):
        pygame.init()
        pygame.mixer.init()
        self.ancho = ancho
        self.alto = alto
        bandera = pygame.FULLSCREEN if pantalla_completa else 0
        self.superficie = pygame.display.set_mode(
            (ancho, alto), bandera
        )
        self.fuente_grande = pygame.font.SysFont('monospace',
52, bold=True)
        self.fuente_normal = pygame.font.SysFont('monospace',
32)
        self.fuente_pequena = pygame.font.SysFont('monospace',
20)
        pygame.mouse.set_visible(False)

```


El método `dibujar_texto()` acepta un parámetro centrado, cuando es `True`, utiliza `Surface.get_rect(center=(x, y))` para calcular automáticamente la posición del texto de forma que quede centrado en la coordenada indicada:

```
def dibujar_texto(self, texto, x, y, color=(255,255,255),
                  fuente='normal', centrado=False):
    fuente_obj = {'grande': self.fuente_grande,
                  'normal': self.fuente_normal,
                  'pequena': self.fuente_pequena}.get(fuente)
    sup = fuente_obj.render(texto, True, color)
    if centrado:
        rect = sup.get_rect(center=(x, y))
        self.superficie.blit(sup, rect)
    else:
        self.superficie.blit(sup, (x, y))
```

7.2 Clase Arranque

El núcleo de la clase `Arranque` es el método `_fade()`. Se calcula el número de pasos en función de la duración y los FPS objetivo. En cada iteración se crea una copia del logo con `set_alpha()` para evitar modificar la superficie original, se limpia la pantalla y se dibuja el logo con la opacidad calculada:

```
def _fade(self, logo, pos_x, pos_y, direccion='in'):
    reloj = pygame.time.Clock()
    pasos = int(self.duracion_fade * 60)
    for i in range(pasos + 1):
        progreso = i / pasos
        if direccion == 'in':
            alpha = int(255 * progreso)
        else:
            alpha = int(255 * (1 - progreso))
        logo_alpha = logo.copy()
        logo_alpha.set_alpha(alpha)
        self.pantalla.limpiar((0, 0, 0))
        self.pantalla.superficie.blit(logo_alpha, (pos_x, pos_y))
        self.pantalla.actualizar()
    reloj.tick(60)
```

7.3 Clase Galeria — loop principal

El loop principal de la galería opera a 30 fps. En cada iteración verifica si hay un USB nuevo (comunicación con `MonitorUSB`), lee la entrada del usuario, procesa la acción y redibuja la pantalla:

```
def ejecutar(self):
    reloj = pygame.time.Clock()
    while self.en_ejecucion:
        if self.usb_monitor.hay_nuevo_usb():
            self._manejar_usb_nuevo()
        accion = self.entrada.leer()
        self._procesar_accion(accion)
        self._dibujar()
        self.pantalla.actualizar()
    reloj.tick(30)
```

7.3 Clase Entrada / método leer()

El método leer() procesa la cola de eventos de pygame y devuelve la primera acción significativa que encuentre. Maneja KEYDOWN, JOYBUTTONDOWN, JOYHATMOTION, JOYAXISMOTION, JOYDEVICEADDED y JOYDEVICEREMOVED. El método _eje_a_accion() aplica el umbral de zona muerta (0.5) al eje 0 (horizontal) y eje 1 (vertical) para convertirlos en acciones de navegación lógicas.

7.3 Clase Emulador / método lanzar()

El método lanzar() implementa el protocolo completo de cesión y recuperación del framebuffer: llama a pygame.display.quit(), espera 1 segundo, lanza mednafen con subprocess.Popen() con las variables SDL en el entorno del subprocesso, espera a que termine con .wait(), luego espera 0.5 s adicionales y reinicializa pygame con display.init() y mixer.init().

8. Integración del subsistema de interfaz

Los tres módulos se integran en el punto de entrada main.py. La instancia sigue un orden determinado por las dependencias: primero crea el objeto Pantalla (que inicializa pygame), luego los demás módulos reciben una referencia a este objeto. El objeto Arranque se instancia con las rutas al logo y al sonido obtenidas del archivo de configuración JSON. Una vez ejecutada la secuencia de arranque, el control se transfiere al loop de Galeria que no termina hasta que el usuario confirma el apagado.

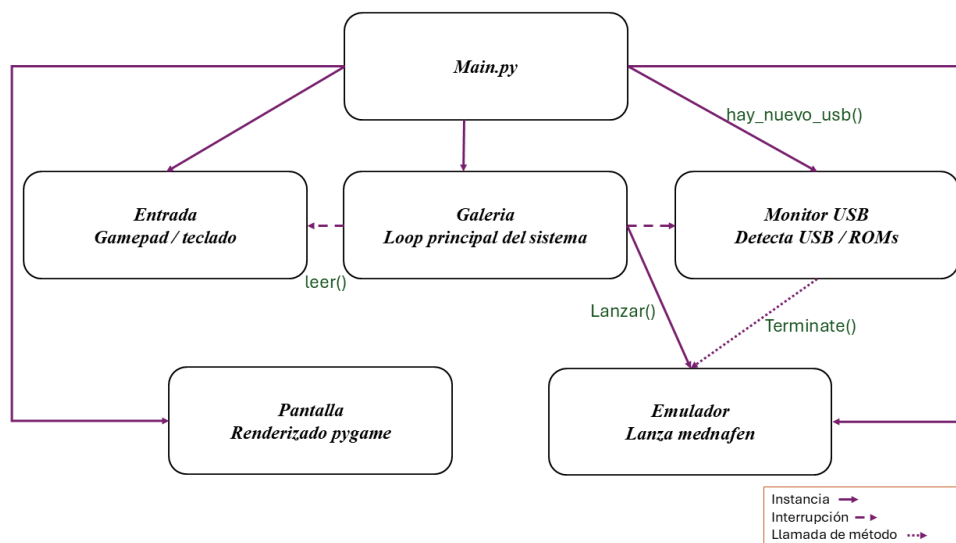


Figura 1. Diagrama de flujo del subsistema de interfaz gráfica

9. Solución de problemas comunes

9.1 Pantalla en negro al iniciar

Causa probable: SDL_VIDEODRIVER no está configurado correctamente. Verificar que la variable de entorno SDL_VIDEODRIVER=kmsdrm está presente en la sesión del servicio systemd. Ejecutar el comando:

```
sudo systemctl status consola-retro.service
```

y revisar si aparece el error “No available video device”. En ese caso, editar el archivo de servicio y agregar la variable de entorno.

9.2 Sin sonido al arrancar

Verificar que el usuario pertenece al grupo audio (groups \$USER) y que `SDL_AUDIODRIVER=alsa` está configurado. También verificar que el volumen de ALSA no está en cero con `alsamixer`.

9.3 `pygame.error: No video mode has been set`

Este error indica que se está intentando dibujar en la superficie antes de que `display.set_mode()` haya sido llamado, o después de que `display.quit()` haya sido invocado. Verificar que la referencia al objeto Pantalla es la misma instancia en todos los módulos.

9.4 Menú con parpadeo visible

Si se observa parpadeo en el menú, verificar que `display.flip()` se llama exactamente una vez por frame al final del ciclo de renderizado, después de que todos los elementos han sido dibujados. Llamadas múltiples a `flip()` en un mismo frame causan parpadeo.

9.5 mednafen no inicia: “No available video device”

Verificar que `pygame.display.quit()` se llama antes de `subprocess.Popen()` y que se espera al menos 1 segundo entre ambas llamadas. También verificar que `SDL_VIDEODRIVER=kmsdrm` está en el entorno del subproceso.

9.6 El gamepad no es detectado al arrancar

Verificar que el usuario pertenece al grupo input con groups \$USER. Si el gamepad se conectó después del arranque, pygame debería detectarlo automáticamente mediante `JOYDEVICEADDED`.

9.7 El USB no es detectado

Verificar que `pyudev` está instalado con `python3 -c 'import pyudev; print(pyudev.__version__)'`. Verificar también que el archivo `/etc/sudoers.d/consola_retro` contiene las reglas de mount y umount sin contraseña, y que el directorio `/media/pi/usb_retro` existe con permisos correctos.

9.8 El sistema no arranca automáticamente

Verificar que el servicio está habilitado con `systemctl is-enabled consola-retro.service`. Verificar que no existe el archivo `/boot/NO_CONSOLA` ni `/boot/firmware/NO_CONSOLA`.

10. Resultados

Repositorio del proyecto: https://github.com/nasaruke/SDK_RetroGames

Video demostrativo: <https://youtu.be/pv0NvIFaCqE>

11. Conclusiones

Se pudo comprobar paso a paso que es posible construir un sistema de entretenimiento digital funcional sin utilizar un entorno gráfico completo. Primero se trabajó con Raspberry Pi OS Lite

Después, se incluyó la biblioteca *pygame* para manejar los gráficos, gracias a ello se mostró el contenido en pantalla sin depender de escritorio. Cabe mencionar que lo primordial, fue configurar el driver `kmsdrm` de `SDL2`, ya que gracias a esto el sistema pudo mostrar la interfaz en pantalla completa desde su arranque.

Durante el desarrollo, también se organizó el código en distintos archivos (*pantalla.py*, *arranque.py* y *galeria.py*). Esta separación ayudó a trabajar de forma más clara, ya que cada parte cumple una función específica, facilitando tanto las pruebas como las mejoras del sistema.

Para una buena experiencia visual, se utilizó la técnica de doble buffer, lo que permitió que las imágenes se mostraran de forma fluida y sin parpadeos. Además, se agregaron elementos como una animación de inicio y sonido, lo cual favorece y enriquece la experiencia. En cuanto al menú, se implementó un desplazamiento que facilita la navegación habiendo muchos elementos.

Finalmente, se configuró el arranque automático mediante *systemd*, lo que permite que todo el sistema funcione desde el momento en que se enciende la Raspberry Pi, sin necesidad de configuraciones manuales.

En conjunto, este proceso muestra de manera clara cómo desarrollar un sistema eficiente, organizado y funcional en un dispositivo con recursos limitados, siguiendo una serie de pasos claramente definidos.

12. Cuestionario

Tabla 4. Cuestionario de evaluación

#	Pregunta
1	¿Por qué se utiliza doble buffer (<code>display.flip</code>) en lugar de actualizar la pantalla píxel a píxel?
2	¿Qué ventaja tiene encapsular todas las llamadas a <code>pygame</code> dentro de la clase <code>Pantalla</code> en lugar de llamar a <code>pygame</code> directamente desde cada módulo?
3	¿Por qué se define un umbral de 0.5 para los ejes analógicos en lugar de detectar cualquier valor distinto de cero? ¿Qué problema resuelve esto en la práctica?
4	Describe el problema del framebuffer compartido entre <code>pygame</code> y <code>mednafen</code> . ¿Por qué es necesario llamar a <code>pygame.display.quit()</code> antes de lanzar <code>mednafen</code> y cómo se recupera la pantalla al regresar?
5	¿Qué función cumple el <code>threading.Lock</code> en la clase <code>MonitorUSB</code> ? ¿Qué condición de carrera podría ocurrir si se eliminara el lock?
6	¿Por qué se utiliza <code>pyudev</code> para detectar la inserción de USB en lugar de revisar periódicamente el contenido de <code>/dev</code> ? Describe las ventajas del enfoque basado en eventos sobre el polling.

13. Bibliografía

[1] E. A. Lee y S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2.a ed. Cambridge, MA: MIT Press, 2017.

[2] Raspberry Pi Foundation, "Raspberry Pi OS Documentation," 2024. [En línea]. Disponible en: <https://www.raspberrypi.com/documentation/>

[3] `pygame` Developers, "pygame Documentation," 2024. [En línea]. Disponible en: <https://www.pygame.org/docs/>

[4] SDL Community, "SDL2 Wiki," 2024. [En línea]. Disponible en: <https://wiki.libsdl.org/SDL2/>

[5] D. J. Greenwalt, "Linux KMS/DRM Architecture," Linux Foundation, 2023. [En línea]. Disponible en: <https://www.kernel.org/doc/html/latest/gpu/drm-kms.html>

[6] Mednafen Team, "Mednafen Documentation," 2024. [En línea]. Disponible en: <https://mednafen.github.io/documentation/>

[7] pyudev Contributors, “pyudev Documentation,” 2024. [En línea]. Disponible en: <https://pyudev.readthedocs.io/>

[8] Python Software Foundation, “subprocess — Subprocess management,” Python 3.11 Documentation, 2024. [En línea]. Disponible en: <https://docs.python.org/3/library/subprocess.html>